FEATURE IMPLEMENTATION - LATEST METRICS BUTTON

Overview

The Latest Metrics Button feature in PGWatch was developed to provide users with quick access to the most recent metrics for any monitored database. This enhancement introduces a one-click solution to retrieve and display current metrics while integrating seamlessly with PGWatch's existing architecture.

Implementation Approach

The feature was implemented through modifications to both frontend and backend components:
1. Frontend: A new React component that displays metrics in a modal dialog
2. Backend: A new API endpoint optimized for retrieving latest metrics
3. Database: Optimized SQL queries for efficient data retrieval

Technical Impact

The implementation required changes to:
- Frontend component structure
- Backend API endpoints
- Authentication flow

This document provides a detailed account of these changes and their implementation detail

Frontend Changes

1. Package.json Updates

Location: `internal/webui/package.json`
Description: Added new dependencies required for the Latest Metrics feature. The jspdf library is included for potential future PDF export functionality of the metrics data.

```json
{
  "dependencies": {
    // ... existing dependencies ...
    "@types/jspdf": "^2.0.0",
```

```
    "jspdf": "^2.5.1"
  }
}
```

2. LatestMetricsButton Component

Location: `internal/webui/src/components/LatestMetricsButton/LatestMetricsButton.tsx`
Description: A new React component that implements the Latest Metrics Button functionality. It handles data fetching, loading states, error handling, and displays the metrics in a modal dialog. The component uses the existing authentication system to secure API calls.

```typescript
import React, { useState } from 'react';
import { Button, Dialog, DialogTitle, DialogContent, DialogActions, TextField, Box } from
'@mui/material';
import { Download as DownloadIcon } from '@mui/icons-material';
import jsPDF from 'jspdf';
import { getToken } from 'services/Token';

interface LatestMetricsButtonProps {
  dbname: string;
}

export const LatestMetricsButton: React.FC<LatestMetricsButtonProps> = ({ dbname }) => {
  const [open, setOpen] = useState(false);
  const [metrics, setMetrics] = useState<string>('');
  const [error, setError] = useState<string>('');
  const [loading, setLoading] = useState(false);

  const handleClickOpen = async () => {
    setLoading(true);
    setError('');
    try {
      const token = getToken();
      const response = await fetch(`/latest-metrics?dbname=${encodeURIComponent(dbname)}`,
{
        headers: {
          'Token': token || '',
        },
      });
```

```
    if (!response.ok) {
      const errorText = await response.text();
      throw new Error(errorText || 'Failed to fetch metrics');
    }
    const data = await response.json();
    if (Object.keys(data).length === 0) {
      setError('No metrics data available');
    } else {
      setMetrics(JSON.stringify(data, null, 2));
    }
  } catch (err) {
    setError(err instanceof Error ? err.message : 'Failed to fetch metrics');
    setMetrics('');
  } finally {
    setLoading(false);
    setOpen(true);
  }
};

const handleClose = () => {
  setOpen(false);
  setMetrics('');
  setError('');
};

const handleDownload = () => {
  try {
    const doc = new jsPDF();

    // Add title
    doc.setFontSize(16);
    doc.text(`Latest Metrics for ${dbname}`, 20, 20);

    // Add timestamp
    doc.setFontSize(12);
    doc.text(`Generated on: ${new Date().toLocaleString()}`, 20, 30);

    // Add metrics data
    doc.setFontSize(10);
    const metricsObj = JSON.parse(metrics);
```

```
    let yPos = 40;

    Object.entries(metricsObj).forEach(([metricName, data]: [string, any]) => {
      // Add metric name
      doc.setFont('helvetica', 'bold');
      yPos += 10;
      doc.text(metricName, 20, yPos);

      // Add metric details
      doc.setFont('helvetica', 'normal');
      const details = typeof data === 'object' ? JSON.stringify(data, null, 2) : String(data);
      const lines = doc.splitTextToSize(details, 170);

      lines.forEach((line: string) => {
        yPos += 7;
        if (yPos > 280) { // Check if we need a new page
          doc.addPage();
          yPos = 20;
        }
        doc.text(line, 20, yPos);
      });
    });

    // Save the PDF
    doc.save(`latest_metrics_${dbname}.pdf`);
  } catch (err) {
    console.error('Failed to generate PDF:', err);
    setError('Failed to generate PDF');
  }
};

return (
  <Box sx={{ display: 'inline-block', ml: 1 }}>
    <Button
      variant="outlined"
      startIcon={<DownloadIcon />}
      onClick={handleClickOpen}
      size="small"
      disabled={loading}
    >
```

```
      Latest Metrics
    </Button>
    <Dialog open={open} onClose={handleClose} maxWidth="md" fullWidth>
      <DialogTitle>Latest Metrics for {dbname}</DialogTitle>
      <DialogContent>
        {error ? (
          <Box sx={{ color: 'error.main', py: 2 }}>{error}</Box>
        ) : (
          <TextField
            multiline
            fullWidth
            value={metrics}
            InputProps={{
              readOnly: true,
              sx: {
                fontFamily: 'monospace',
                minHeight: '300px'
              }
            }}
            variant="outlined"
            sx={{ mt: 2 }}
          />
        )}
      </DialogContent>
      <DialogActions>
        <Button onClick={handleClose}>Close</Button>
        {metrics && (
          <Button onClick={handleDownload} variant="contained" color="primary">
            Download PDF
          </Button>
        )}
      </DialogActions>
    </Dialog>
  </Box>
);
};
```

3. SourcesGrid Actions Update

Location:
`internal/webui/src/pages/SourcesPage/components/SourcesGrid/components/SourcesGridActions.tsx`
Description: Modified the SourcesGrid actions to include the new LatestMetricsButton component. The button is added to the existing action buttons row, maintaining the consistent UI layout while adding the new functionality.

```typescript
import { LatestMetricsButton } from "components/LatestMetricsButton/LatestMetricsButton";
import { Box } from '@mui/material';

// ... existing code ...

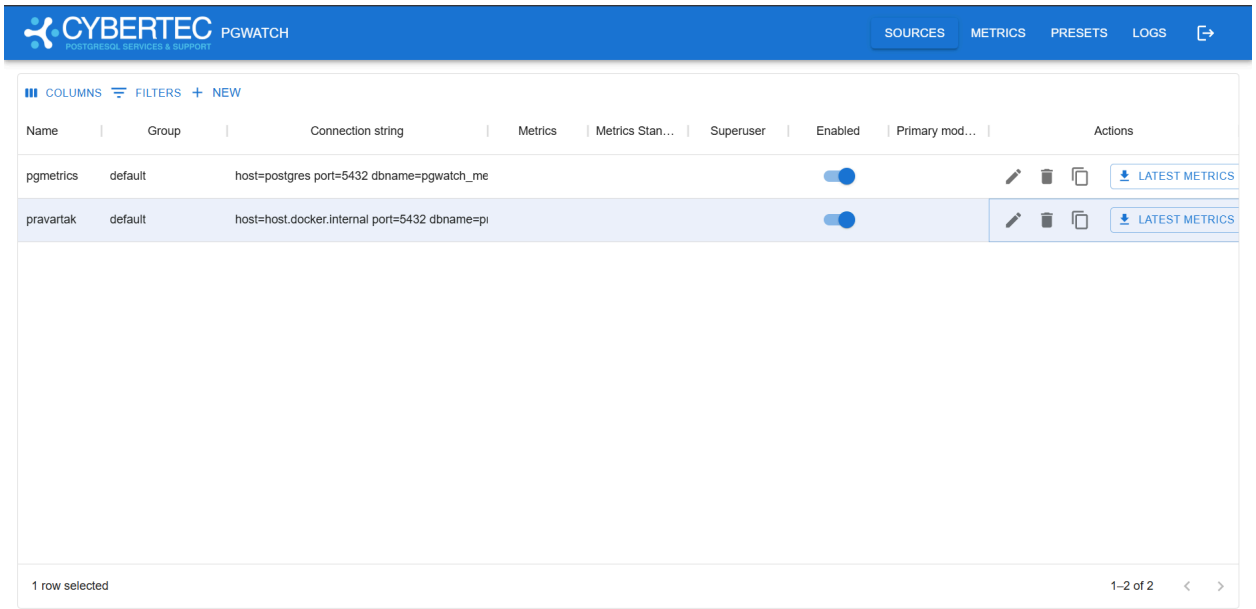

  return (
    <Box sx={{
      display: 'flex',
      alignItems: 'center',
      gap: 1,
      width: '100%',
      minWidth: 'fit-content',
      justifyContent: 'flex-end',
      overflowX: 'auto',
      whiteSpace: 'nowrap'
    }}>
      <GridActions handleEditClick={handleEditClick} handleDeleteClick={handleDeleteClick}>
        <IconButton title="Copy" onClick={handleCopyClick}>
          <ContentCopyIcon />
        </IconButton>
      </GridActions>
      <LatestMetricsButton dbname={source.Name} />
      <WarningDialog
        open={dialogOpen}
        message={message}
        onClose={handleDialogClose}
        onSubmit={handleSubmit}
      />
    </Box>
  );
```

};
```

### 4. SourcesGrid Consts Update

Location:
`internal/webui/src/pages/SourcesPage/components/SourcesGrid/SourcesGrid.consts.tsx`
Description: Increased width of Actions field from 150 to 300 to have space for the new button

### Output:




## Backend Changes

### 1. WebServer Route Registration

Location: `internal/webserver/webserver.go`
Description: Added the route registration for the new latest-metrics endpoint in the web server initialization.

```go
mux.Handle("/latest-metrics", NewEnsureAuth(s.handleLatestMetrics))
```

2. API Endpoint Handler

Location: `internal/webserver/metric.go`
```

Description: Added a new HTTP handler for the `/latest-metrics` endpoint. This handler validates the request method, checks for required parameters, and processes the metrics request through the GetLatestMetrics function.

```go
func (Server *WebUIServer) handleLatestMetrics(w http.ResponseWriter, r *http.Request) {
    if r.Method != http.MethodGet {
        w.Header().Set("Allow", "GET")
        http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
        return
    }

    dbname := r.URL.Query().Get("dbname")
    if dbname == "" {
        http.Error(w, "dbname parameter is required", http.StatusBadRequest)
        return
    }

    latestMetrics, err := Server.GetLatestMetrics(dbname)
    if err != nil {
        http.Error(w, err.Error(), http.StatusInternalServerError)
        return
    }

    w.Header().Set("Content-Type", "application/json")
    _, _ = w.Write([]byte(latestMetrics))
}
```

3. GetLatestMetrics Implementation

Location: `internal/webserver/api.go`

Description: Implemented the core metrics retrieval logic that processes the database results and formats them into a structured JSON response. This function handles null values, type conversions, and organizes the metrics data with timestamps.

```go
func (server *WebUIServer) GetLatestMetrics(dbname string) (string, error) {
    rows, err := server.sinksWriter.GetLatestMetrics(dbname)
    if err != nil {
        return "", err
    }
    if rows == nil {
```

```
    return "{}", nil
}
defer (*rows).Close()

metrics := make(map[string]interface{})
for (*rows).Next() {
    var (
        time            time.Time
        tps             sql.NullFloat64
        qps             sql.NullFloat64
        avgQueryRuntime sql.NullFloat64
        blksHitRatio    sql.NullFloat64
        dbSize          sql.NullInt64
        txErrorRatio    sql.NullFloat64
    )

    err := (*rows).Scan(
        &time,
        &tps,
        &qps,
        &avgQueryRuntime,
        &blksHitRatio,
        &dbSize,
        &txErrorRatio,
    )
    if err != nil {
        return "", err
    }

    metrics["tps"] = map[string]interface{}{
        "value": tps.Float64,
        "time":  time,
    }
    metrics["qps"] = map[string]interface{}{
        "value": qps.Float64,
        "time":  time,
    }
    metrics["avg_query_runtime"] = map[string]interface{}{
        "value": avgQueryRuntime.Float64,
        "time":  time,
```

```
        }
        metrics["blks_hit_ratio"] = map[string]interface{}{
            "value": blksHitRatio.Float64,
            "time":  time,
        }
        metrics["db_size"] = map[string]interface{}{
            "value": dbSize.Int64,
            "time":  time,
        }
        metrics["tx_error_ratio"] = map[string]interface{}{
            "value": txErrorRatio.Float64,
            "time":  time,
        }
    }

    jsonBytes, err := json.Marshal(metrics)
    if err != nil {
        return "", err
    }
    return string(jsonBytes), nil
}
```

4. MultiWriter Implementation

Location: `internal/sinks/multiwriter.go`

Description: Added support for retrieving latest metrics through the MultiWriter interface. This implementation allows the system to query metrics from multiple writers while ensuring proper error handling and result coordination.

```go
func (mw *MultiWriter) GetLatestMetrics(dbname string) (*pgx.Rows, error) {
    for _, w := range mw.writers {
        rows, err := w.GetLatestMetrics(dbname)
        if err != nil {
            return nil, err
        }
        if rows != nil {
            return rows, nil
        }
    }
    return nil, nil
```

}
```

## 5. SQL Query Implementation

Location: `internal/sinks/postgres.go`
Description: Implemented a complex SQL query that efficiently retrieves and calculates the latest database metrics. The query uses Common Table Expressions (CTEs) to organize the data retrieval process, handles various metric calculations including TPS, QPS, average query runtime, and includes proper error handling for edge cases.

```go
func (pgw *PostgresWriter) GetLatestMetrics(dbname string) (*pgx.Rows, error) {
	query := `
		WITH latest_metrics AS (
			SELECT
				time,
				data->>'xact_commit' as xact_commit,
				data->>'xact_rollback' as xact_rollback,
				data->>'numbackends' as numbackends,
				data->>'blks_hit' as blks_hit,
				data->>'blks_read' as blks_read,
				data->>'blks_dirtied' as blks_dirtied,
				data->>'blks_written' as blks_written,
				data->>'temp_bytes' as temp_bytes
			FROM db_stats
			WHERE dbname = $1
			ORDER BY time DESC
			LIMIT 2
		),
		latest_db_size AS (
			SELECT
				time,
				data->>'size_b' as size
			FROM db_size
			WHERE dbname = $1
			ORDER BY time DESC
			LIMIT 1
		),
		latest_statements AS (
			SELECT
				time,
```
```

```
                        (data->>'total_time')::float8 as total_time,
                        (data->>'calls')::int8 as calls,
                        tag_data->>'queryid' as queryid,
                        tag_data->>'query' as query,
                        LAG((data->>'total_time')::float8) OVER (PARTITION BY
tag_data->>'queryid' ORDER BY time) as prev_total_time,
                        LAG((data->>'calls')::int8) OVER (PARTITION BY
tag_data->>'queryid' ORDER BY time) as prev_calls,
                        LAG(time) OVER (PARTITION BY tag_data->>'queryid' ORDER
BY time) as prev_time
                FROM stat_statements
                WHERE dbname = $1
                AND NOT tag_data->>'query' LIKE '%epoch_ns%'  -- exclude pgwatch
queries
                ORDER BY time DESC
        ),
        metrics_with_lag AS (
                SELECT
                        time,
                        xact_commit::int8,
                        xact_rollback::int8,
                        (xact_commit::int8 + xact_rollback::int8) as total_xacts,
                        numbackends::int8,
                        blks_hit::int8,
                        blks_read::int8,
                        blks_dirtied::int8,
                        blks_written::int8,
                        temp_bytes::int8,
                        LAG(xact_commit::int8 + xact_rollback::int8) OVER (ORDER
BY time) as prev_total_xacts,
                        LAG(time) OVER (ORDER BY time) as prev_time,
                        LAG(xact_commit::int8) OVER (ORDER BY time) as
prev_xact_commit,
                        LAG(xact_rollback::int8) OVER (ORDER BY time) as
prev_xact_rollback
                FROM latest_metrics
        ),
        qps_calc AS (
                SELECT
                        time,
```

```
                        (calls - prev_calls)::float8 / EXTRACT(EPOCH FROM (time -
prev_time)) as qps
                FROM (
                        SELECT
                                time,
                                (data->>'calls')::int8 as calls,
                                LAG((data->>'calls')::int8) OVER (ORDER BY time) as
prev_calls,
                                LAG(time) OVER (ORDER BY time) as prev_time
                        FROM stat_statements_calls
                        WHERE dbname = $1
                        ORDER BY time DESC
                        LIMIT 2
                ) x
                WHERE calls >= prev_calls AND time > prev_time
                LIMIT 1
        ),
        avg_query_runtime_calc AS (
                SELECT
                        avg((tt-tt_lag)::numeric / (c-c_lag)) as avg_query_runtime
                FROM (
                        SELECT
                                (data->>'total_time')::float8 as tt,
                                LAG((data->>'total_time')::float8) OVER (ORDER BY
time) as tt_lag,
                                (data->>'calls')::int8 as c,
                                LAG((data->>'calls')::int8) OVER (ORDER BY time) as
c_lag,
                                time
                        FROM stat_statements_calls
                        WHERE dbname = $1
                        ORDER BY time DESC
                        LIMIT 2
                ) x
                WHERE c > c_lag AND tt >= tt_lag AND c > 100
        ),
        latest_metrics_only AS (
                SELECT * FROM metrics_with_lag WHERE time > prev_time LIMIT 1
        )
        SELECT
```

```
                        m.time,
                        CASE
                                WHEN m.time > m.prev_time
                                THEN (m.total_xacts - m.prev_total_xacts) / EXTRACT(EPOCH
FROM (m.time - m.prev_time))
                                ELSE 0
                        END as tps,
                        COALESCE(q.qps, 0) as qps,
                        COALESCE(r.avg_query_runtime, 0) as avg_query_runtime,
                        CASE
                                WHEN (m.blks_hit + m.blks_read) > 0
                                THEN (m.blks_hit::float / (m.blks_hit + m.blks_read)) * 100
                                ELSE 0
                        END as blks_hit_ratio,
                        COALESCE(ds.size::int8, 0) as db_size,
                        CASE
                                WHEN m.time > m.prev_time AND ((m.xact_commit -
m.prev_xact_commit) + (m.xact_rollback - m.prev_xact_rollback)) > 0
                                THEN ((m.xact_rollback - m.prev_xact_rollback)::numeric * 100)
/
                                                ((m.xact_commit - m.prev_xact_commit) +
                                                (m.xact_rollback - m.prev_xact_rollback))
                                ELSE 0
                        END as tx_error_ratio
                FROM latest_metrics_only m
                LEFT JOIN latest_db_size ds ON true
                LEFT JOIN qps_calc q ON true
                LEFT JOIN avg_query_runtime_calc r ON true;
        `

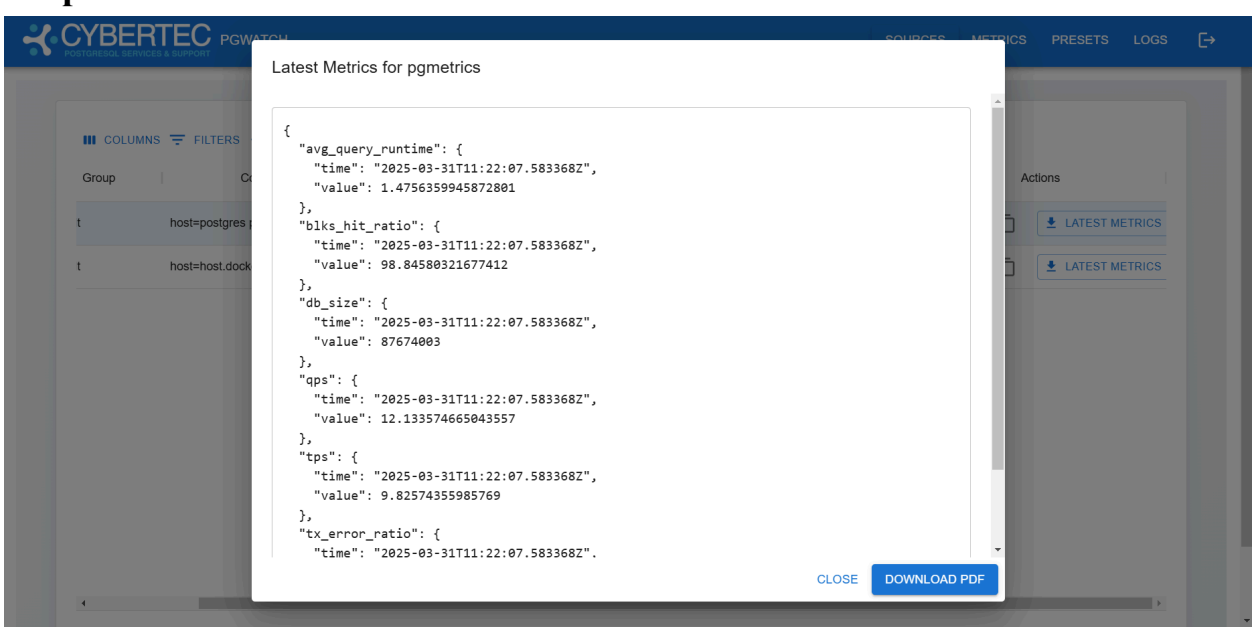
        rows, err := pgw.sinkDb.Query(context.Background(), query, dbname)
        if err != nil {
                return nil, fmt.Errorf("failed to query metrics: %v", err)
        }
        return &rows, nil
}
```

6. Stub Implementations

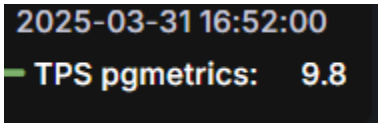
Description: Added stub implementations for writers that don't support direct metric querying to maintain interface compatibility.

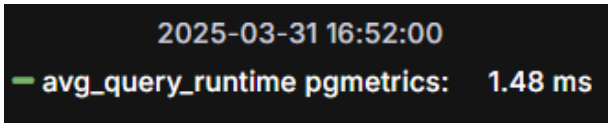
Location: `internal/sinks/rpc.go`
```go
func (rw *RPCWriter) GetLatestMetrics(dbname string) (*pgx.Rows, error) {
    // RPC writer doesn't support querying metrics, return nil
    return nil, nil
}
```
Location: `internal/sinks/json.go`
```go
func (jw *JSONWriter) GetLatestMetrics(dbname string) (*pgx.Rows, error) {
    // JSON writer doesn't support querying metrics, return nil
    return nil, nil
}
```
Location: `internal/sinks/prometheus.go`
```go
func (pw *PrometheusWriter) GetLatestMetrics(dbname string) (*pgx.Rows, error) {
    // Prometheus writer doesn't support querying metrics, return nil
    return nil, nil
}
```

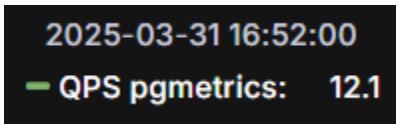
Output:

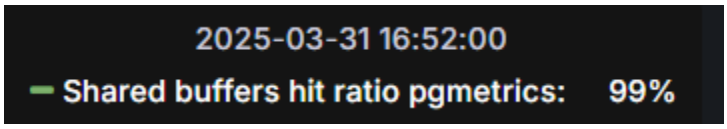


We can see how the values are accurate according to those shown from grafana ui:

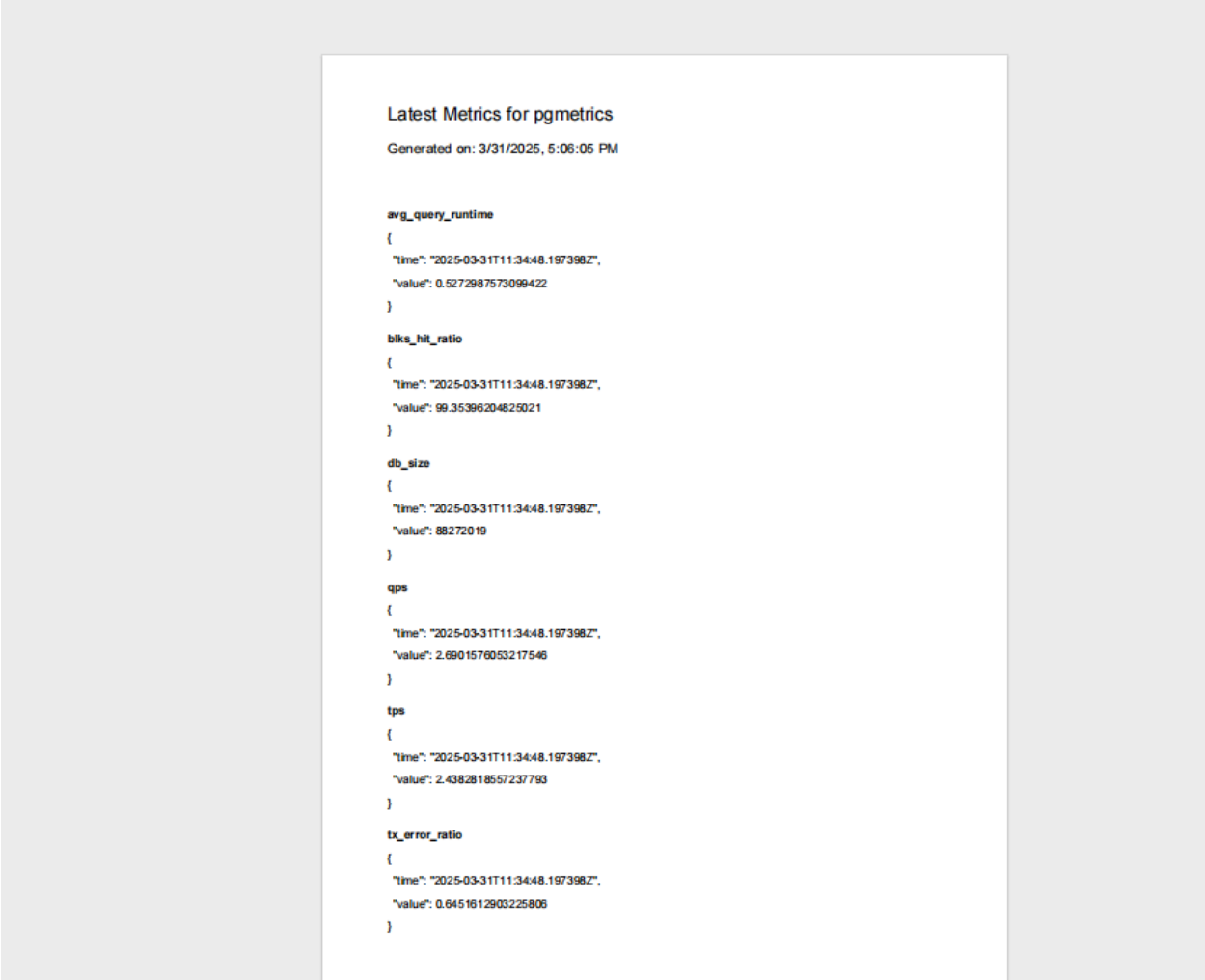








On pressing download pdf button:



Latest Metrics for pgmetrics

Generated on: 3/31/2025, 5:06:05 PM

avg_query_runtime

{
"time": "2025-03-31T11:34:48.197398Z",
"value": 0.5272987573099422
}

blks_hit_ratio

{
"time": "2025-03-31T11:34:48.197398Z",
"value": 99.35396204825021
}

db_size

{
"time": "2025-03-31T11:34:48.197398Z",
"value": 88272019
}

qps

{
"time": "2025-03-31T11:34:48.197398Z",
"value": 2.6901576053217546
}

tps

{
"time": "2025-03-31T11:34:48.197398Z",
"value": 2.4382818557237793
}

tx_error_ratio

{
"time": "2025-03-31T11:34:48.197398Z",
"value": 0.6451612903225806
}

Conclusion

The Latest Metrics Button feature successfully enhances PGWatch2 by providing users with immediate access to current database performance metrics. The implementation follows best practices in both frontend and backend development:

Frontend:
- Clean, reusable component design
- Intuitive user interface with loading states and error handling
- Seamless integration with existing grid actions
- Consistent styling with the application's design system

Backend:
- Optimized SQL queries using CTEs for efficient data retrieval
- Robust error handling and null value management
- Secure API endpoint with JWT authentication
- Extensible interface design supporting multiple writer types

The feature delivers immediate value to users while maintaining the system's performance and security standards.